Nama	:	Ifan Prima Nursaid
NIM	:	231351063
Kelas	:	Malam A

Kode sumber (source code) untuk semua program yang dibahas dalam laporan ini dapat diakses secara publik melalui repositori GitHub pribadi saya pada tautan berikut:

https://github.com/ifanprima130206/Tugas-Komputasi-Paralel

1. Tugas 4: Integrasi Numerik

integrasi_numeric.py

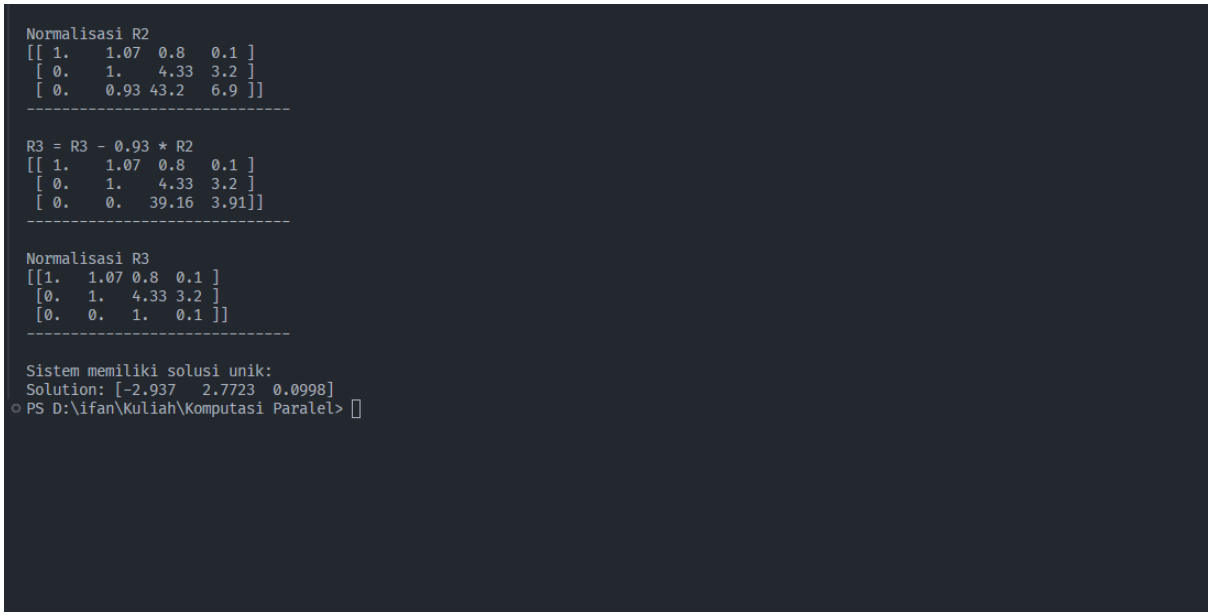
```python
import numpy as np

def f(x):
        return x**2

a = int(input("Masukan batas bawah (a): "))
b = int(input("Masukan batas atas (b): "))

n = int(input("Masukan jumlah partisi (n): "))

h = (b - a) / n

x = np.linspace(a, b, n+1)
y = f(x)
integral = (h/2) * (y[0] + 2 * sum(y[1:-1]) + y[-1])

print("Hasil integrasi numerik: ", integral)
```

codesnap.dev

```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\integrasi_numerik.py
Masukan batas bawah (a): 10
Masukan batas atas (b): 20
Masukan jumlah partisi (n): 5
Hasil integrasi numerik:  2340.0
```

2. Tugas 5: Sistem Persamaan Linear

```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\sistem_persamaan_linear.py
Masukkan jumlah persamaan: 3
Masukkan jumlah variabel: 3
Masukkan matriks ukuran 3 x 4:
Baris 1: 1 2 44 7
Baris 2: 2 8 27 19
Baris 3: 30 32 24 3

Original Matrix:
[[ 1.  2. 44.  7.]
 [ 2.  8. 27. 19.]
 [30. 32. 24.  3.]]
------------------------------

R1 <-> R3
[[30. 32. 24.  3.]
 [ 2.  8. 27. 19.]
 [ 1.  2. 44.  7.]]
------------------------------

Normalisasi R1
[[ 1.    1.07  0.8   0.1 ]
 [ 2.    8.   27.   19.  ]
 [ 1.    2.   44.    7.  ]]
------------------------------

R2 = R2 - 2.00 * R1
[[ 1.    1.07  0.8   0.1 ]
 [ 0.    5.87 25.4  18.8 ]
 [ 1.    2.   44.    7.  ]]
------------------------------

R3 = R3 - 1.00 * R1
[[ 1.    1.07  0.8   0.1 ]
 [ 0.    5.87 25.4  18.8 ]
 [ 0.    0.93 43.2   6.9 ]]
------------------------------

Normalisasi R2
[[ 1.    1.07  0.8   0.1 ]
 [ 0.    1.    4.33  3.2 ]
 [ 0.    0.93 43.2   6.9 ]]
------------------------------

R3 = R3 - 0.93 * R2
[[ 1.    1.07  0.8   0.1 ]
 [ 0.    1.    4.33  3.2 ]
 [ 0.    0.   39.16  3.91]]
------------------------------

Normalisasi R3
```




```
import numpy as np
def get_matrix_input():
while True:
try:
n = int(input("Masukkan jumlah persamaan (jumlah baris matriks): "))
m = int(input("Masukkan jumlah variabel (jumlah kolom matriks sebelum
kolom hasil): "))
if n <= 0 or m <= 0:
print("Jumlah persamaan dan variabel harus lebih dari 0.")
continue
matrix = np.zeros((n, m + 1))
print(f"Masukkan elemen-elemen matriks ({n}x{m+1}). Pisahkan dengan
spasi:")
for i in range(n):
while True:
try:
row_input = input(f"Baris {i+1}: ").split()
if len(row_input) != m + 1:
print(f"Masukkan {m+1} angka untuk baris ini. Coba lagi.")
continue
matrix[i] = [float(x) for x in row_input]
break
```

```
except ValueError:
print("Input tidak valid. Masukkan hanya angka.")
return matrix
except ValueError:
print("Input tidak valid. Masukkan hanya angka untuk jumlah persamaan
dan variabel.")
matrix = get_matrix_input()
def gaussian_elimination(matrix, rows, cols):
matrix = np.array(matrix, dtype=float)
print("\nOriginal Matrix:")
print(np.around(matrix, 2))
print("-" * 30)
for i in range(rows):
pivot_row = i
for k in range(i + 1, rows):
if abs(matrix[k, i]) > abs(matrix[pivot_row, i]):
pivot_row = k
if pivot_row != i:
matrix[[i, pivot_row]] = matrix[[pivot_row, i]]
print(f"\nR{i+1} <-> R{pivot_row+1}:")
print(np.around(matrix, 2))
print("-" * 30)
pivot_element = matrix[i, i]
if pivot_element == 0:
print(f"\nWarning: Pivot element at row {i+1}, column {i+1} is zero. "
f"The matrix may be singular or have infinite solutions.")
continue
matrix[i, :] = matrix[i, :] / pivot_element
print(f"\nR{i+1}/{pivot_element:.2f}:")
print(np.around(matrix, 2))
print("-" * 30)
for k in range(i + 1, rows):
factor = matrix[k, i]
matrix[k, :] = matrix[k, :] - factor * matrix[i, :]
```

```
print(f"\nR{k+1} = R{k+1} - {factor:.2f} * R{i+1}:")
print(np.around(matrix, 2))
print("-" * 30)
return matrix
# Ambil ukuran matriks
rows, cols = matrix.shape
modified_matrix = gaussian_elimination(matrix, rows, cols)
print("\nMatrix Setelah eliminasi Gaussian:")
print(np.around(modified_matrix, 2))
rank_coefficient = np.linalg.matrix_rank(modified_matrix[:, :-1])
rank_augmented = np.linalg.matrix_rank(modified_matrix)
num_variables = cols - 1
if rank_coefficient != rank_augmented:
print("\nSistem tidak memiliki solusi.")
elif rank_coefficient == rank_augmented:
if rank_coefficient == num_variables:
print("\nPersamaan memiliki solusi yang unik.")
solution = np.zeros(num_variables)
for i in range(num_variables - 1, -1, -1):
solution[i] = modified_matrix[i, -1]
for j in range(i + 1, num_variables):
solution[i] -= modified_matrix[i, j] * solution[j]
solution[i] /= modified_matrix[i, i]
print("Solution:", solution)
else:
print("\nPersamaan memiliki solusi infinite.")
```

3. Tugas 6: Kriptografi Hash Cipher

```
PS D:\ifan\Kuliah\Komputasi Paralel> python .\kriptografi_hash_cipher.py
Masukkan pesan: Hallo
Masukkan password: Minimal8@

Cipher (hex): e8b589dc6b
Hasil dekripsi: Hallo
PS D:\ifan\Kuliah\Komputasi Paralel>
```

integrasi_numeric.py

```python
import hashlib
import binascii


def derive_keystream(key: bytes, length: int) -> bytes:
    keystream = b""
    counter = 0

    while len(keystream) < length:
        counter_bytes = counter.to_bytes(4, "big")
        block = hashlib.sha256(key + counter_bytes).digest()
        keystream += block
        counter += 1

    return keystream[:length]


def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))


def encrypt(message: str, password: str) -> str:
    msg_bytes = message.encode("utf-8")
    key_bytes = password.encode("utf-8")

    keystream = derive_keystream(key_bytes, len(msg_bytes))
    cipher_bytes = xor_bytes(msg_bytes, keystream)

    return binascii.hexlify(cipher_bytes).decode("ascii")


def decrypt(cipher_hex: str, password: str) -> str:
    cipher_bytes = binascii.unhexlify(cipher_hex)
    key_bytes = password.encode("utf-8")

    keystream = derive_keystream(key_bytes, len(cipher_bytes))
    plain_bytes = xor_bytes(cipher_bytes, keystream)

    return plain_bytes.decode("utf-8")


if __name__ == "__main__":
    pesan = input("Masukkan pesan: ")
    kunci = input("Masukkan password: ")

    cipher = encrypt(pesan, kunci)
    print("\nCipher (hex):", cipher)

    recovered = decrypt(cipher, kunci)
    print("Hasil dekripsi:", recovered)
```